COP3530 Project 2

MiniDB

November 2026

Administrative Information

Team name	Oracle (Group 94)
Team members	Zhengmao Zhang (`billzhengmaozhang`) Jacob Lindland (`jdljake`) My Pham (`phammy237`)
GitHub repository	billzhengmaozhang/COP3530p2
Video demo	Link to be added when the recording is ready

1. Extended and Refined Proposal

Problem and motivation. MiniDB is an in-memory database for stock trades. It answers exact-ID queries and compares two ways of finding the same trade: a custom separate-chaining hash table and a custom B+ tree. Searching an unsorted collection can require inspecting every row. An index avoids a full record scan, but it takes work to build and maintain. The project asks how much that work costs and whether the answer changes with dataset size, insertion order, or whether a queried ID exists.

Implemented features. The terminal menu loads and saves trade CSV files, inserts a validated trade, and looks up a trade by ID using the index selected by the user. Successful insertion updates both indexes. Duplicate IDs are rejected and a failed CSV load preserves the active dataset. A successful load replaces the active dataset. The benchmark menu generates synthetic data, compares the two indexes in isolation, prints median timings, and saves per-trial results as CSV.

Data. Each row has a positive 64-bit trade ID, a ticker symbol, a price in integer cents, a positive share quantity, and a nonnegative Unix timestamp in seconds. Storing prices in cents preserves their exact values. The generator assigns contiguous unique IDs and draws the remaining fields from fixed distributions using a seed. It can emit rows in ascending ID order or shuffle those same rows. The repository includes a small synthetic sample CSV. The default benchmark generates 100,000, 250,000, and 500,000 trades in both orders.

Tools and data structures. MiniDB uses C++17 and CMake. Catch2 v3.5.4 and CTest support the B+ tree tests. An append-only `std::vector` owns the full records. Both indexes implement `IIndex` and store only a trade ID and a zero-based row position. The position stays valid when the vector reallocates. Lookups return an optional position so that row zero and a missing ID are distinct. The C++ standard library provides vectors, smart pointers, binary search, pseudorandom generation, file streams, and `std::chrono::steady_clock` for timing.

1. Extended and Refined Proposal (continued)

Hash table. The hash table starts with 16 buckets. A 64-bit integer mixer spreads keys across the buckets. A vector within each bucket holds colliding entries, which are searched by exact ID. Before an insertion would push the load factor above 0.75, the implementation doubles the bucket count and redistributes the entries.

B+ tree. The default maximum is 32 children per internal node. Binary search over separators chooses a child, with equal keys routed to the right. A second binary search locates the ID in the leaf. Leaves hold sorted keys and their row positions, and a link points to the next leaf. An overflowing leaf splits and copies its right half's first key into the parent. An overflowing internal node promotes a separator. Splits can propagate to the root and create a new root, keeping all leaves at the same depth.

Benchmark procedure. For each size and order, the runner times building an empty index, inserting $\max(1, \lfloor n/10 \rfloor)$ more keys, finding existing keys, and finding missing keys. The lookup workloads each use 100,000 queries by default. The two indexes receive identical keys and alternate which runs first. Each configuration uses one warmup and five measured trials by default. The runner verifies returned positions and compares checksums, with structural checks outside the timed regions where applicable. The benchmark runs independently of the active database.

Distribution of responsibility. The proposal assigned Zhengmao Zhang the hash table, collision and resize testing, and hash benchmark integration. Jacob Lindland was assigned the B+ tree, its insertion and split tests, and tree benchmark integration. My Pham was assigned record storage, data generation, CSV I/O, the menu, and the benchmark runner. The repository records Jacob's implementation of the B+ tree and its Catch2 tests, and Zhengmao's integration of the CLI, database, and benchmark components.

2. Analysis

Changes from the proposal. The proposal described generating synthetic CSV datasets. The benchmark menu generates its datasets directly in memory, which lets users run the suite without preparing input files and keeps CSV parsing outside the measurements. The menu focuses on exact-ID lookup so the comparison uses the same operations on both indexes. The B+ tree retains linked leaves for a possible range-query extension. CSV export follows the design's write-then-rename approach and uses a unique staging directory so separate saves cannot share a temporary file.

2. Analysis (continued)

Worst-case running time. Let n be the number of indexed records, m the number of rows imported, b the number of bytes in the input file, and q the number of lookups in a batch. Assume a fixed B+ tree node capacity and the default hash-table configuration.

Operation	**Worst case**	**Reason**
Hash lookup	$O(n)$	One bucket can contain all keys.
Hash insertion	$O(n)$	Scanning a chain or rehashing can take linear time.
B+ tree lookup	$O(\log n)$	Binary search at each level of a balanced tree.
B+ tree insertion	$O(\log n)$	A split may propagate up one root-to-leaf path.
Database insertion	$O(n)$	Hashing or growing the record vector can take linear time.
CSV load	$O(b + m^2)$	Read and parse the file, then build both indexes.
CSV save	$O(n)$	Validate and write each record once.
Dataset generation	$O(n)$	Generate each row and optionally shuffle the vector.

Database lookup inherits the selected index's bound because resolving the row position and copying a bounded-size record takes constant time. A well-distributed hash gives expected $O(1)$ lookup and expected amortized $O(1)$ insertion. Building from empty takes $O(n^2)$ in the worst case for hashing and $O(n \log n)$ for the B+ tree. An incremental insert batch adds $\Theta(n)$ keys and has the same bounds. The timed lookup batches take $O(qn)$ and $O(q \log n)$, respectively. Outside timing, the hash structure check can also take $O(n^2)$ because it scans earlier entries in each bucket for duplicates. The record store and both indexes together use $O(n)$ space under the default configuration.

Measured results. Each cell shows hash / B+ tree median nanoseconds per operation from the project benchmark summary. Build time is divided by the number of records built.

Rows	Order	Build	Insert	Hit	Miss
100,000	Sequential	181.68 / 47.84	78.97 / 50.41	17.20 / 102.96	15.97 / 13.09
100,000	Shuffled	166.65 / 177.78	64.40 / 145.94	14.46 / 92.47	12.46 / 11.64
250,000	Sequential	181.72 / 49.10	113.86 / 58.10	19.78 / 132.10	21.17 / 14.85
250,000	Shuffled	163.74 / 203.98	89.10 / 166.52	19.34 / 117.31	19.64 / 12.43
500,000	Sequential	200.08 / 51.08	121.56 / 55.03	26.89 / 162.36	23.23 / 14.85
500,000	Shuffled	195.14 / 227.95	125.66 / 183.37	29.47 / 148.55	25.55 / 15.77

2. Analysis (continued)

Interpretation. The hash table was faster on successful lookups in all six configurations. At 500,000 shuffled records it took 29.47 ns per hit versus 148.55 ns for the tree. The B+ tree built faster and handled the additional inserts faster for all three sequential datasets. The hash table led both workloads for all three shuffled datasets. Increasing IDs send B+ tree inserts to the rightmost leaf and append within it. Shuffling sends inserts across different leaves and can require shifting entries within a leaf. These differences are consistent with the measurements.

The B+ tree was faster on misses in this run. The benchmark generates missing IDs above the largest stored ID and the additional insertion range. That makes all tree misses follow the same rightmost path, which can keep the visited nodes in cache. The result describes this out-of-range workload. Misses spread across the key space would exercise a different access pattern.

3. Reflection

Project experience and challenges. The project combined implementing the indexes with designing a way to compare them. The shared interface made it possible to use one benchmark for both implementations. The B+ tree exposed a subtle correctness issue to test: when a leaf splits, searching for the promoted separator must still find the record in the leaf on the right. Small node capacities made it possible to force leaf, internal, and root splits with manageable test sequences. Another challenge was distinguishing an indexing failure from an input failure. Keeping the old database until a CSV import has passed validation gave the application a clear rule for handling malformed rows and duplicate IDs.

What we would change. We would add shared index tests and a CSV-to-lookup smoke test early so every component has the same acceptance criteria. We would also benchmark absent IDs between stored keys and archive each run with its revision and build settings.

What each member learned. Zhengmao Zhang, Jacob Lindland, and My Pham each learned the importance of agreeing on the architecture early, assigning clear responsibilities, and communicating as work progresses. MiniDB's shared index interface illustrates why those habits matter. Both indexes had to return a row position for the same trade ID so the database could retrieve the record and the benchmark could compare the results. Clear ownership gives each member a component to validate, and regular communication helps catch mismatched assumptions before integration.

References

[1] Oracle (Group 94), *MiniDB Database Indexing Proposal.*

[2] MiniDB, *Component Design and Interface Specification.* `docs/MiniDB_Design.md`.

[3] MiniDB, implementation and tests.

[4] MiniDB, archived benchmark trials.